

Lec 04 Optimizers - Part B

Lec 04 Optimizers - Part B

Lecture	04 / 63 + intro
Duration	34:13
Captions	English (auto-generated)
Words	5,368
Video	7ymbACnFrb4
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone, welcome to the lecture series on fundamentals of generative VI and the large language models. In our previous session, if you remember, we have started with the optimizers where we have seen the four different optimizers and in this particular session, we will continue with the remaining optimizers. So let's quickly recall what we have learned in the previous. So is it not the weight update is stochastic gradient descent with momentum is what we had done and then we had stopped the previous session. So before we start the next I want you to look into this particular example. So here stochastic gradient descent I'm just taking only the weight update formula here w_{new} is equal to w_{old}

[00:48] minus learning rate derivative of loss with respect to weight. So G uh if you substitute G with this then you you it is same as whatever the formula that we have seen in the previous session and from the previous we have also learned this when the gradient is a positive when the gradient is a positive weight gets decreased when the gradient is a negative weight gets increased. This we have seen it right. You can just take the values substitute in this formula and then you can see when the gradient is a positive the weight has decreased. When the gradient is negative the weight has increased. So this is what we have seen.

[01:30] Now I would also uh give little importance on what if the gradient magnitude is large. So what if the gradient is a large gradient? Look at this. I'm now not talking about a positive, negative or anything. What if the gradient is very large gradient then you see uh substitute the values in this and um you is it not there is a too much uh um you know there is a large change in the weight earlier old weight was five now if the gradient is very large value there is too much of update so from five now the weight is four so is it not the value the weight value has changed measured by a large amount here.

[02:18] Um so from five directly it has come down to four. So there so if the gradient is large what is happening there will be um a visible good change in the weight at the same time you look at this when the gradient is a small gradient. Look at this how much we have changed from five you have changed to a small value from five you have come down to old weight was five now the new weight is 4.95 whereas when the gradient is large from five you have changed it to four from five here we have changed it to 4.95 what what is that we have observed if the gradient is a positive weight is decreasing if the gradient is negative the weight is increasing and if The gradient is very large, update is large.

[03:07] If the gradient is small, update is small. Right? So this is what we have understood from this slide. What are we trying to understand from this? So uh and if you observe in each of these the learning rate is common learning rate is same. So here the gradient sign gradient sign decides the direction whereas gradient magnitude decides how much weight has to change. Is it not? This is what we have understood from this slide. I hope this is clear uh to all of us. Yes. Okay. Now let us uh uh take uh uh one scenario. What is that scenario? uh in the previous slide the weights was uh uh the learning rate was kept common now I'm changing the learning rate uh uh weight is old weight is five the gradient is 10 which means

[04:06] I'm now talking about a scenario where the gradient is a large gradient but what is that you have to focus here look at the learning rate is different can you look into this um old weight is five learning rate what you have taken is a smaller learning rate and you see there is a small change in the weight okay now take a moderate learning rate substitute the values in this and then you see there is a change from 5 to 4 but look at this last one there is a large gradient learning rate is high when learning rate is high is it not too How much abrupt change has occurred from five directly? We have made the weight as zero. It's something scary, right?

[04:59] It's not something very usual one. Okay. What are we trying to understand? Learning rate has an impact, right? Okay. Let's now take a uh other side where the gradient is small. Okay. Again, we are seeing the weights are uh I'm sorry the learning rates are different. Look at this learning rate. Look at this learning rate. Look at this learning rate. Okay. When we have a smaller learning rate, there is a small uh uh change in the weight. Okay. When the moderate uh uh when you are

using a moderate learning rate, there is uh okay when there is a high learning rate, larger learning rate, you see here.

[05:44] So I'm not asking you to focus on if the gradient is a large or gradient is small. I'm only asking you to focus on uh um based on different learning rate there is an impact on weight update. Agree to this? Yes. So for the same gradient whether it's large or small gradient not bothered. learning a rate actually controls how the weight actually changes. I hope you are able to convinced you are convinced with this particular uh slide about this uh statement that is learning rate controls how the weight actually changes. Okay.

[06:27] Now let's uh uh uh come back to one important point in a neural network. It is not just one weight or a few weights. You will have millions of weights and all those weights will not receive the same gradient. Do you agree to this? Uh all weights will not receive uh the gradients of same magnitude. Some will receive a larger gradient. Some will receive a smaller gradient. So when uh the weights may receive some weights may receive a larger gradients and some weights may receive a small or rare gradients. Then uh uh stoastic gradient descent what we did we have used a same learning array to throughout but that's not an ideal scenario right when I have a larger gradient when I have a smaller gradient how can I have a same weight

[07:18] update do you agree to this point are you able to get this uh intuition now look at this particular scenario if a parameter is already receiving a large gradient which means you see here large gradient Now for that by suppose if you apply a larger learning rate if you have a larger learning rate what is happening there is an adverse effect right uh so in this scenario ideally what is the correct one learning rate should be small then only the uh you will have uh this impact will not be there are you able to get this uh some weights will receive larger gradients some weights will receive a smaller gradients Suppose if a weight if a if a weight has received a larger gradient and there by chance if a model has assumed an higher

[08:12] learning rate then adverse effect the weight has fallen to zero weight has from old weight to new weight it is 5 to zero this is totally an acceptable one in this case ideally what would have been good if the learning rate would have been a smaller learning rate probably we wouldn't have seen this zero Maybe whatever may be the value but at least zero will not be there. At the other side look at this scenario where the parameter is already receiving a smaller gradient. We have a small gradient here and in for this already the gradient is small in that we have put a small learning rate. Right? What is happening? The weight update is very very slow. Is it not? How much you have changed it from five you have changed it

[08:58] to only 4.99. How much impact it will have on the error? Very small impact. Is it not convergence will take so much of time reaching to global minima will be uh time consuming in this case. Ideally what should be the scenario? The scenario

should be the learning rate should be large. The learning rate should be large here. Are you getting this concept? So here what exactly we are trying to understand is when parameters are getting different uh gradients ideally using the same learning rate is not an correct solution. Uh now you can ask why uh same solution uh same learning rate because stochastic gradient descent assumes the same learning rate throughout but we don't want that based on if the learning

[09:46] rate is uh uh if the gradient is very large I want the learning rate to be small if the gradient is small I want the model to assume a larger uh learning rate so this scenario is good right now you can ask once again you may get this particular Question ma'am what if why are we considering only the current scenario? How about considering over a period of time from the past if all the gradients what we have obtained is large then cumulative sum is also very large.

[10:19] If that is the case, can we have a smaller learning rate? Just not for the current one, we got smaller gradient. But over a past, we have observed that all the gradients are small. In that case, confidently we can increase the uh learning rate. Is this scenario is good? Are you able to get this intuition? You can pause, you can rewind, you can see, you can listen to this once again. What I'm trying to explain is just by looking at the current gradient uh deciding smaller learning rate or larger learning rate if this is not an ideal scenario how about this scenario just don't decide based on the current decide based on the past from the past if all the gradients are large then confidently increase the

[11:03] learning rate I mean decrease the learning rate if the past all the gradients are very small then cumulative sum also will be smaller When cumulative sum also is small then you can confidently increase the learning rate. Is the scenario good? If this scenario is good then we need a different optimizer. That optimizer is adaptive gradient adagrad. Adaptive gradient. Now uh so how does this adaptive gradient adrad works? This is a normal uh uh optimizer. So far what optimizes we have seen W_{new} is equal to W_{old} minus learning rate derivative of loss with respect to weight here this derivative of loss with respect to weight and uh this learning rate is um nothing to do with this I'm talking about uh learning

[11:53] rate this learning rate whether it's a gradient is large small whatever we have not bothered the learning rate was kept small I mean kept the same for all the weights this is a scenario which we don't want we want to avoid this scenario if that is the case can I have a different learning rate for uh different weights can we have this answer is yes we we have seen this with one example in the previous but at the same time we don't want to just because for the current uh uh samples the gradient is small so let me increase the weight just for the current gradient is large let me reduce the weight no I don't want to decide based on the current one if the past from the past experience if all of those are small

[12:43] then I get the confidence that yes confidently we can go ahead and increase is it not this kind of a scenario which means we now need accumulative sum of all the gradients. So cumulative sum can be taken this way uh adagrad keeps track of sum of square of past gradients. So how do you take a sum of square of past gradients? Is it not this summation is a sum g_i is uh this derivative of loss with respect to weight which is a gradient and then you are taking a square and then you are taking all of the previous instance. So from $I = 1$ till if you're doing a current just based on current you are not deciding it you're deciding based on all its previous. So till the previous take the square and then uh add it. So at every

[13:35] step take the gradient square it add it to the previous total. So what is this uh t minus one α t minus one is actually doing it is accumulated squared uh gradients it is from the previous step. accumulated squared gradients from the previous step is what this α t minus one is actually giving us. Okay. Now if that is the case what is that weight update in adagrad? So the weight update in adagrad is $w_{\text{new}} = w_{\text{old}} - \text{learning rate} \times \text{derivative of loss with respect to weight}$ or learning rate gt .

[14:17] This is what we had seen so far. But now I don't want to keep a same learning rate for all um the weights. So how you have to change this learning rate? What is the formula that you have changed that learning rate to? You see the learning rate is uh [laughter] adrad uses the learning new learning rate. So it is initial learning rate ϵ α minus under the square root α t minus 1 + some constant epsilon. So this uh constant um you are writing only to ensure denominator should not become zero. Okay. Now uh okay let us see there is some formula. Now you can see how this formula will help in achieving whether to increase the way uh learning rate or decreasing the learning rate.

[15:06] Let us try to understand that uh scenario. Now uh so what we have understood uh so far same learning rate we don't want the same learning rate to happen. If that is the case based on the gradient we want to increase or decrease but at the same time just based on the current we don't want to take the decision. Therefore we have taken the cumulative sum of the square of all the past gradients. Okay. Now how based on that we are going to get different learning rates. Now you tell me what does this uh u uh this particular uh holds is it not um okay uh this is nothing but from the previous it is the sum of square of all the past gradients that is what is this part okay now you tell me if uh if this holds sum of

[15:59] square of all the past gradients if all the previous are small uh uh let us assume all the previous one for the first uh u mini uh uh batch let us assume the gradient is 0.1 for the uh second the gradient is 0.2 for the third one for the third um batch uh the gradient what we have obtained is uh 0.3 let us assume this now what are we taking we are taking the square of this okay let's take this also why only positive Let us assume 0.4. We have got this. We are taking the square of this.

[16:40] Taking the square of this. Taking the square of this. And you are adding each of these. Is it not? When all the previous gradients are small, square and cumulative uh sum also will be small. If the cumulative sum is small then is it not denominator term in this entire denominator term is small. When denominator term is small what what will be this value? Just think back when denominator something divided by uh something when the denominator value is very small. Is it not we are going to get a larger value? High value not larger high value then is it not the learning rate is high here? Are you getting this? Yes. Okay. Now see uh when the uh gradients are very large uh say for example the gradient is five when uh

[17:37] the gradient is 5.95 then we have minus4 then we have three then we have 0.79 we have like this the gradients where the gradients are very large then take the square of it and then take the sum of it is it not this Entire result value will be very high. Which value is high? Is it not alpha? T minus one value is high. Where is that alpha? Denominator. If denominator is very high, what will be this value? Is it not small? So is it not? When the cumulative sum of all the previous gradients are very large, weight has to be uh learning rate has to be uh decreased. If the cumulative sum of all of the previous gradients are very small, then denominator term is very small. When denominator term is very small, then this value is

[18:31] increasing. Is it not your uh um changing the learning rate, learning rate is increasing here. So what are we trying to understand here? So this is what if the parameter is receiving a small gradients and its accumulated sum also will remain small. when this is uh uh small then the denominator value is small. If denominator value is small what is happening to um this particular term this is high relatively large. So this means adagrad has allowed a larger learning rates for those parameters which is having a smaller uh gradients.

[19:08] Are you getting this? Yes. Okay. Similar way vice versa is um the opposite. What if the gradients are very large? We want smaller learning rate. How we are we achieving a smaller learning rate? If this is very large, if denominator is very large, automatically this is small. So we are achieving uh an different learning rate based on the gradients. So this is not possible with any of those conventional optimizers. This is adagrat.

[19:37] There's one drawback with adagrat. What is that a drawback? Look at this. It is an accumulator sum of square of all the gradients. Now when the cumulative sum of squares of all the gradients what if this value is very large uh by doing this what will happen if it is very very uh large value can you think of ultimately the weight is w_{new} is equal to w_{old} minus the learning rate derivative of loss with respect to weight now I'm saying that this value is very very large value because the cumulative sum we have no control when we have a deeper neural Will it work?

[20:16] What if the gradients are very large value? Then what is that we are doing?

The the learning rate has to be uh reduced when uh this is very large value. Numerator denominator increases then automatically learning rate is reduced when learning rate is uh small uh uh over a period of time. What exactly is going to happen? Um is it not with a smaller η learning say for example this value is 25 uh just for an example and then you are taking a smaller learning uh uh rate let us assume you're taking a smaller learning rate. Now when you multiply this with 25 the resultant value is very less this entire value is very less. If this entire value is less is it not old value will be equivalent to new value. Are you

[21:08] getting this? If this is the case, when do you converge? Uh uh um converging to a global minima will be very very uh slow. This what I have explained here. Um this keep decreasing over uh time. Uh uh with the time definitely this value keeps decreasing. Um then uh uh at one point of time the learning rate becomes too small. When learning rate becomes too small then the formula W_{new} is equal to W_{old} . Yeah, take this formula. Uh over a period of time if this entire term becomes very very small then you are left out with W_{new} is almost similar to the previous weight only new weight is similar to previous weight. If new weight is similar to the previous one then there is no impact on the loss. Right? This is the problem

[22:02] with adagrad. So the whole problem has occurred because of this term. Right? Because of this term uh in the adagrad this whole problem has occurred. Can we do some change here? Answer is yes. That change is rms propagation root mean square propagation. The next optimizer. So the problem has occurred because you were taking the cumulative uh sum. So we don't want that. Instead what we are doing once again we are going back to that momentum term. Can you see this exponentially weighted moving average.

[22:38] So uh at every step compute the gradient. No doubt even in the previous also we computed the gradient. Now also you compute the gradient. But instead of taking the cumulative sum can you take uh the moving uh uh average exponentially um moving uh weighted moving average. Can we do this? This is the formula. Can we do this? Is it not something similar to what we have already seen? Uh V_T is βV_T minus one plus $1 - \beta$. Then we have derivative of loss with respect to that current weight. We have seen this something similar already in SDDD with the momentum. So is it not something similar you are applying here also in RMS propagation. What we are doing? We are calculating exponentially weighted

[23:29] moving average. Uh but we are still taking the square of it. Okay. Can we just see one numerical example how how this is having an impact? What exactly is exponentially moving average? So what what is RMS propagation? This is the weight update. Uh this is the learning rate and this is the weight update. What is the new learning rate here? having a different learning rate is a good concept from uh SGD I mean from stochastic gradient descent with the momentum we can definitely take that particular uh concept but uh the problem there is when the gradient is um you know

over a period of time when that cumulative sum increases and over a period of time that learning rate is decreasing and old weight is equivalent

[24:16] to new weight to avoid that some portion of it uh you you're now taking an exponential weighted moving average. We are writing it as ST. Okay, ST stands for exponentially weighted moving average of squared gradients at time t. Okay, so uh now you instead of $\alpha t - 1$ at the bottom uh we are now taking so if this is $\alpha t - 1$ this will remain $t - 1$. Yeah. So instead of taking this uh uh here we are taking s. So what is ST? Squared uh exponentially weighted moving average of squared gradient. So we have taken ST here. I hope uh this is uh clear to you.

[25:03] Now let us try to understand uh with one numerical example what this uh exponentially weighted how previous gradients are given less importance how current gradient is given more importance all that we will understand with one example. So if beta value is 0.9 is it not $0.9 s_{t-1} + 0.1 g_t^2$ okay look at uh this uh this is what is an um example we are considering now at step one or at a time instance one can you see here definitely $t - 1$ previous instance we have still not so not will be zero right so when you substitute the values in as is it not we got this uh uh equ value s_1 is 0.1 g_1 squared right uh nothing I have done I've just substituted the values what is that you're trying to understand

[26:01] I'm trying to explain you what is that exponentially weighted moving average um you know squared gradient I'm explaining that term here now for the next iteration or for the next instance is it not the formula is $0.9 s_{t-1} + 0.1 g_t$ uh time is uh t uh when time when t equal to 2 $t - 1$ is 1 and this is g_2 so is it not this formula okay just substitute the values in this what is s_1 s_1 is this so can you substitute the values in this what are we understanding from here what is exponentially weighted moving average uh uh consider the previous gradients that will help us in understanding certain things but don't give too much importance Current gradient should be given higher importance if that is the concept here.

[26:53] What is the current gradient? Is it not the current gradient is G_2 ? Is it not we have given more importance to the current gradient but at the same time previous also will help me in understanding something. So that the previous how much importance we are giving less importance we are giving for this uh previous gradient. So this is what is RMS propagation. If you are still not getting this, let's go to the next uh uh iteration. So here it is s_3 where t equal to 3. Substitute the values. Is it not $0.9 s_{t-1}$? Then $0.1 g_3$ squared. Just substitute the values back into it. What is s_{t-1} ? s_{t-1} is this formula. Uh this is the equation.

[27:36] So substitute that equation. If there are any um typos uh you can rectify uh correct yourself. So um by doing this is it not we got this uh uh value by multiplying this with this is it not we got this and what is the current gradient here the current

gradient is G_3 but we are just not considering the current we are considering the previous also but we are giving less importance in the decreasing importance we are giving to the past gradients comparing to G_1 G_2 is given more importance G_1 is little older now so we are giving less less importance to the older gradient then this importance and high importance is given to the current gradient. So older gradients G_1 square now has less influence than G_2 and G_3 .

[28:27] So uh are we getting uh this concept right? So similar way we have G_4 you can see G_1 is given uh relatively very less than G_2 then G_3 and the current gradient is given an higher importance. So here we can observe um as you when you are in the current gradient there is an um less importance. Okay there is um less importance for all of its previous uh gradient. What are we understanding uh from this? This is what is this exponentially weighted moving average.

[29:04] So it is you are considering the current gradient but in order to perform something on the current gradient you are taking the past gradients and uh why did we come to RMS propagation because having different learning rates for each of these weight updates is good but problem with adagrad is at one point old weight is coming equal to new weight. We want to avoid that but having different learning rate concept is good. So what are we doing because of this we are now problem is happening only because of this concept otherwise rest everything was good with adagrad. So we have slightly tweaked only that part. So how how how is that we have tweaked we have taken the exponentially weighted uh moving

[29:50] average of squared gradients. We are taking this rest everything will remain the same. And from this what are your understanding is how exactly that uh uh definition of um you know the previous should be given current should be given more importance. Previous you should consider but you should give less importance. The this particular whole slide is explaining you that concept. For the first time current gradient is given complete importance for for the next along with the current gradient you are giving considering the previous also but you have given less importance for the next instance current is given more importance less less this is further less and for the next current gradient is given high importance this is less

[30:37] this is further less this is further less okay so uh this is what is RMS uh propagation But otherwise rest everything will remain the same. Go back and then uh think what will be this ST value. If this ST value is small, denominator value will be small. If denominator value is small, the learning rate will be high. If this is very large value then denominator is large then this value will be small then the learning rate will be small. So adagrad and RMS propagation you are achieving uh different learning rates for different weights. This concept has been achieved but adagrad problem is overcome using RMS propagation. And the last optimizer that we have is Adam. Uh this is the uh last optimizer uh for this particular uh

[31:29] session but although there are many new optimizers which has come up. uh so some of the fundamental optimizers in that uh Adam. So what is Adam? It is the now so far if you see what all different optimizers that we have discussed uh gradient descent stoastic gradient descent mini batch stoastic gradient then stoastic gradient with momentum that momentum uh concept is very nice because you are uh you're not too much fluctuating you are able to consider uh the exponential one and you are able to move smoothly then we have seen adagrad and then RMS propagation adagrad is having a problem but RMS propagation is fine. So now what is Adam? They have taken the advantage of both these two optimizers combined and then they have

[32:18] come up with the new optimizer. Therefore Adam is the combination of momentum with RMS propagation. These two are um combined together is Adam. So um you see is it not this is the momentum term if you recall from our uh since in both the places there are beta for momentum term I'm using beta 1 for RMS propagation beta I'm using beta 2 is it not these two is what we have seen in our session so far momentum term is this and here it is a squared of gradient so by combining these two this is Adam can you see where learning rate this is a momentum term and this is an RMS propagation term. So this is all the different optimizers. With this we have come to an end of uh this particular session. So far what we have uh

[33:14] discussed we have first started we are in the week one. We have first started with understanding the fundamental terms that are used in generative uh uh modeling or generative VI concepts. Then we have moved to different uh uh activation functions, different loss functions that are used in a neural network. And then in the last session and this particular session is mainly focused on the optimizers which play a key role in updating the weights. So with this whatever the fundamental concepts that was needed for our upcoming weeks, we have come to an end.

[33:51] In the next two sessions, we are going to study a quick recap of a convolutional neural network. So we will understand in detail about how each of these uh uh layers work. That will be the topic of discussion for our next two sessions. With this thank you.